Session 26 — LIVE CODE Walkthrough

This code-first guide follows the authoritative classroom implementation sequence exactly. Apply each step in order; file paths, code, commands, and identifiers are reproduced unchanged.

LC-01 — CORE: the real `Issue` type grows its new fields

Step: `LC-01`

GOAL

The frontend's own `Issue` type matches every real field the backend already returns.

TECHNICAL CONTEXT

What new problem are you solving? The real instructor-provided backend support already returns `priority` / `storyPoints` / `sprintId` on every real issue -- but the frontend's own `Issue` type does not know about any of it yet. Updating the real shared type first, before any real component uses it, is what lets every later step's own TypeScript checking actually catch a real mistake.

FILE

`web/src/lib/types.ts`

EXACT LOCATION

Inside the real `Issue` type, right after `status`, before the existing `assigneeId` field.

BEFORE

TSX

```
  status: string;
  assigneeId: number | null;
```

IMPLEMENTATION

TSX

```
  status: string;
  priority: string;
  storyPoints: number | null;
  sprintId: number | null;
  assigneeId: number | null;
```

AFTER

TSX

```
  status: string;
  priority: string;
  storyPoints: number | null;
  sprintId: number | null;
  assigneeId: number | null;
```

WHY THIS WORKS

`priority` / `status` are plain real strings, never a second real enum type -- the identical real convention this app already uses, since the real backend itself validates both as plain strings, not a TypeScript/Prisma enum.

VERIFY

BASH

```
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. Nothing uses the real new fields yet.

LC-02 — CORE: the real `Sprint` frontend type

Step: `LC-02`

GOAL

A real `Sprint` type exists for later use, matching exactly what the real backend's Sprint endpoints already return.

TECHNICAL CONTEXT

What new problem are you solving? The real instructor-provided backend support already returns a whole real Sprint resource -- but nothing on the frontend knows its shape yet. Adding the real type now, before any real component uses it, is what lets TEAMWORK's own Sprint work start from an already-accurate shape instead of guessing it from the real API as it goes.

FILE

`web/src/lib/types.ts`

EXACT LOCATION

A real new `Sprint` type is added right after the real `Issue` type's closing brace.

BEFORE

TSX

```
  reporterId: number;
  createdAt: string;
};
```

IMPLEMENTATION

TSX

```
  reporterId: number;
  createdAt: string;
};

export type Sprint = {
  id: number;
  projectId: number;
  name: string;
  goal: string;
  status: string;
  startedAt: string | null;
  completedAt: string | null;
  createdAt: string;
  points: {
    issueCount: number;
    totalPoints: number;
    completedPoints: number;
    completionPercent: number;
  };
};
```

AFTER

TSX

```
  reporterId: number;
  createdAt: string;
};

export type Sprint = {
  id: number;
  projectId: number;
```

```
  name: string;
  goal: string;
  status: string;
  startedAt: string | null;
  completedAt: string | null;
  createdAt: string;
  points: {
    issueCount: number;
    totalPoints: number;
    completedPoints: number;
    completionPercent: number;
  };
};
```

WHY THIS WORKS

`points` is a real nested object, never four flat top-level fields -- it mirrors exactly what the real backend's own Sprint response nests, so no frontend code ever has to re-derive it.

VERIFY

BASH

```
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. Nothing uses `Sprint` yet (TEAMWORK).

LC-03 — CORE: `createIssueAction` -- the real validation skeleton

Step: `LC-03`

GOAL

A real new export exists that reads a real issue-creation form and validates it, before it ever calls the real backend.

TECHNICAL CONTEXT

What new problem are you solving? Every real mutation this app has built so far (login, register, create workspace) uses the identical real Server-Action-form shape: validate, call the real backend, handle the real error, redirect/return. Create Issue is the real next instance of that same shape -- proving it again, on a new real resource, is the point.

FILE

`web/src/lib/actions/issues.ts`

EXACT LOCATION

A brand-new file, in the existing `lib/actions/` folder.

BEFORE

File does not exist yet.

IMPLEMENTATION

TSX

```tsx
"use server";

import { apiFetch } from "@/lib/api";
import { requireSessionToken } from "@/lib/session";

export interface IssueFormState {
  error?: string;
}

export async function createIssueAction(
  projectId: number,
  basePath: string,
  _prevState: IssueFormState,
  formData: FormData,
): Promise<IssueFormState> {
  const token = await requireSessionToken();
  const title = String(formData.get("title") ?? "").trim();
  const description = String(formData.get("description") ?? "").trim();

  if (!title) {
    return { error: "Title is required." };
  }

  // The real backend call comes next (LC-04).
  return {};
}
```

AFTER

TSX

```tsx
"use server";

import { apiFetch } from "@/lib/api";
import { requireSessionToken } from "@/lib/session";

export interface IssueFormState {
  error?: string;
}

export async function createIssueAction(
  projectId: number,
  basePath: string,
  _prevState: IssueFormState,
  formData: FormData,
): Promise<IssueFormState> {
  const token = await requireSessionToken();
  const title = String(formData.get("title") ?? "").trim();
  const description = String(formData.get("description") ?? "").trim();

  if (!title) {
    return { error: "Title is required." };
  }

  // The real backend call comes next (LC-04).
  return {};
}
```

WHY THIS WORKS

`projectId` and `basePath` arrive as real plain leading arguments, bound via `.bind()` at the real call site (LC-05) -- the identical real pattern this action's own two leading parameters need, since neither one is a real form field a student types.

VERIFY

BASH

```bash
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. Nothing calls the real backend yet (LC-04).

LC-04 — CORE: `createIssueAction` -- the real backend call

Step: `LC-04`

GOAL

A real new issue is actually created against the real backend, and the real creator lands on its own new Issue Detail page.

TECHNICAL CONTEXT

Pure, mechanical continuation of LC-03 -- the real validation already exists; this step only calls the real `POST /issues` route (already real and working since an earlier session) and redirects to the real result.

FILE

`web/src/lib/actions/issues.ts`

EXACT LOCATION

The `import` list grows by one; the final `return {};` placeholder is replaced by the real backend call.

BEFORE

TSX

```tsx
import { apiFetch } from "@/lib/api";
```

TSX

```tsx
  // The real backend call comes next (LC-04).
  return {};
}
```

IMPLEMENTATION

TSX

```tsx
import { redirect } from "next/navigation";
import { apiFetch } from "@/lib/api";
```

TSX

```tsx
  const res = await apiFetch("/issues", {
    method: "POST",
    token,
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ projectId, title, description }),
```

```
  });

  if (!res.ok) {
    return { error: "Something went wrong. Please try again." };
  }

  const issue: { id: number } = await res.json();
  redirect(`${basePath}/issues/${issue.id}`);
}
```

AFTER

TSX

```
import { redirect } from "next/navigation";
import { apiFetch } from "@/lib/api";
```

TSX

```
  const res = await apiFetch("/issues", {
    method: "POST",
    token,
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ projectId, title, description }),
  });

  if (!res.ok) {
    return { error: "Something went wrong. Please try again." };
  }

  const issue: { id: number } = await res.json();
  redirect(`${basePath}/issues/${issue.id}`);
}
```

WHY THIS WORKS

Landing on the real new issue's own Issue Detail page (not back on the list) is the identical real post-creation UX `createWorkspaceAction` already established -- a student should see the real thing they just made, not have to find it themselves.

VERIFY

BASH

```
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. Nothing calls `createIssueAction` from anywhere yet (LC-05).

LC-05 — CORE: `CreateIssueForm` -- the real action wiring

Step: `LC-05`

GOAL

A real form shell exists, wired to `createIssueAction` through `useActionState` -- no real fields yet.

TECHNICAL CONTEXT

The identical real `useActionState` shape every other form-action this app has built already uses (`CreateWorkspaceForm`, `RegisterForm`) -- proving the pattern transfers to a third real resource.

FILE

`web/src/app/(app)/workspaces/[id]/projects/[projectId]/issues/new/CreateIssueForm.tsx`

EXACT LOCATION

A brand-new file at `.../issues/new/CreateIssueForm.tsx`, with no real fields yet.

BEFORE

File does not exist yet.

IMPLEMENTATION

TSX

```tsx
"use client";

import { useActionState } from "react";
import { createIssueAction, type IssueFormState } from "@/lib/actions/issues";

const initialState: IssueFormState = {};

export function CreateIssueForm({
  projectId,
  basePath,
}: {
  projectId: number;
  basePath: string;
}) {
```

```
  const action = createIssueAction.bind(null, projectId, basePath);
  const [state, formAction, pending] = useActionState(action, initialState);

  return <form action={formAction} className="flex flex-col gap-3" />;
}
```

AFTER

TSX

```
"use client";

import { useActionState } from "react";
import { createIssueAction, type IssueFormState } from "@/lib/actions/issues";

const initialState: IssueFormState = {};

export function CreateIssueForm({
  projectId,
  basePath,
}: {
  projectId: number;
  basePath: string;
}) {
  const action = createIssueAction.bind(null, projectId, basePath);
  const [state, formAction, pending] = useActionState(action, initialState);

  return <form action={formAction} className="flex flex-col gap-3" />;
}
```

WHY THIS WORKS

`createIssueAction.bind(null, projectId, basePath)` -- the identical real way `CreateWorkspaceForm` would bind extra real arguments ahead of the two `useActionState` itself always supplies (`prevState` , `formData`); `projectId` / `basePath` are never real form fields.

VERIFY

BASH

```
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. The real title field, error display, and submit button come next (LC-06).

LC-06 — CORE: `CreateIssueForm` -- the real title field and submit

Step: `LC-06`

GOAL

The real form now collects a real title, shows a real backend error, and offers a real way to submit.

TECHNICAL CONTEXT

Pure, mechanical continuation of LC-05 -- the identical real field/error/submit shape every other form-action in this app already uses.

FILE

`web/src/app/(app)/workspaces/[id]/projects/[projectId]/issues/new/CreateIssueForm.tsx`

EXACT LOCATION

The self-closing `<form />` grows real children: an error paragraph, the Title field, and a submit button.

BEFORE

TSX

```
  return <form action={formAction} className="flex flex-col gap-3" />;
```

IMPLEMENTATION

TSX

```
  return (
    <form action={formAction} className="flex flex-col gap-3">
      {state.error && (
        <p className="text-sm text-red-600 dark:text-red-400">{state.error}</p>
      )}
      <label className="flex flex-col gap-1 text-sm">
        Title
        <input
          name="title"
          type="text"
          required
          className="rounded-md border border-gray-300 px-3 py-1 text-sm dark:border-gray-
600 dark:bg-gray-800"
```

```
        />
      </label>
      <button
        type="submit"
        disabled={pending}
        className="rounded-md border border-blue-600 bg-blue-600 px-3 py-1 text-sm font-
medium text-white disabled:opacity-50"
      >
        {pending ? "Creating..." : "Create issue"}
      </button>
    </form>
  );
```

AFTER

TSX

```
  return (
    <form action={formAction} className="flex flex-col gap-3">
      {state.error && (
        <p className="text-sm text-red-600 dark:text-red-400">{state.error}</p>
      )}
      <label className="flex flex-col gap-1 text-sm">
        Title
        <input
          name="title"
          type="text"
          required
          className="rounded-md border border-gray-300 px-3 py-1 text-sm dark:border-gray-
600 dark:bg-gray-800"
        />
      </label>
      <button
        type="submit"
        disabled={pending}
        className="rounded-md border border-blue-600 bg-blue-600 px-3 py-1 text-sm font-
medium text-white disabled:opacity-50"
      >
        {pending ? "Creating..." : "Create issue"}
      </button>
    </form>
  );
```

WHY THIS WORKS

`disabled={pending}` -- a real double-submit (e.g. a fast real double-click) must not fire a second real `createIssueAction` call while the first is still in flight.

VERIFY

BASH

```
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. The real description field comes next (LC-07).

LC-07 — CORE: `CreateIssueForm` -- the real description field

Step: `LC-07`

GOAL

The real form also collects an optional description.

TECHNICAL CONTEXT

Pure, mechanical continuation of LC-06 -- one more real field, the identical real shape.

FILE

`web/src/app/(app)/workspaces/[id]/projects/[projectId]/issues/new/CreateIssueForm.tsx`

EXACT LOCATION

Right after the real Title `<label>`, before the submit `<button>`.

BEFORE

TSX

```
      </label>
      <button
        type="submit"
```

IMPLEMENTATION

TSX

```
      </label>
      <label className="flex flex-col gap-1 text-sm">
        Description (optional)
        <textarea
          name="description"
          rows={3}
          className="rounded-md border border-gray-300 px-3 py-1 text-sm dark:border-gray-
600 dark:bg-gray-800"
        />
      </label>
      <button
        type="submit"
```

AFTER

TSX

```
      </label>
      <label className="flex flex-col gap-1 text-sm">
        Description (optional)
        <textarea
          name="description"
          rows={3}
          className="rounded-md border border-gray-300 px-3 py-1 text-sm dark:border-gray-
600 dark:bg-gray-800"
        />
      </label>
      <button
        type="submit"
```

WHY THIS WORKS

A `<textarea>`, not an `<input>` -- the real backend's own `description` field (already real since an earlier session) is free text that can genuinely run longer than one line.

VERIFY

BASH

```
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. Nothing renders this component yet (LC-08).

LC-08 — CORE: the real `/issues/new` page

Step: `LC-08`

GOAL

A real, visitable page exists that renders `CreateIssueForm`.

TECHNICAL CONTEXT

Pure wiring -- the real form already exists (LC-05/06); this step only gives it a real route.

FILE

`web/src/app/(app)/workspaces/[id]/projects/[projectId]/issues/new/page.tsx`

EXACT LOCATION

A brand-new file at `.../issues/new/page.tsx`, a real sibling of `CreateIssueForm.tsx`.

BEFORE

File does not exist yet.

IMPLEMENTATION

TSX

```tsx
import { Card } from "@/components/ui/Card";
import { CreateIssueForm } from "./CreateIssueForm";

export default async function NewIssuePage({
  params,
}: {
  params: Promise<{ id: string; projectId: string }>;
}) {
  const { id, projectId } = await params;
  const basePath = `/workspaces/${id}/projects/${projectId}`;

  return (
    <main className="mx-auto flex max-w-sm flex-col gap-4 p-6">
      <Card>
        <h1 className="mb-4 text-xl font-semibold">Create an issue</h1>
```

```
        <CreateIssueForm projectId={Number(projectId)} basePath={basePath} />
      </Card>
    </main>
  );
}
```

AFTER

TSX

```tsx
import { Card } from "@/components/ui/Card";
import { CreateIssueForm } from "./CreateIssueForm";

export default async function NewIssuePage({
  params,
}: {
  params: Promise<{ id: string; projectId: string }>;
}) {
  const { id, projectId } = await params;
  const basePath = `/workspaces/${id}/projects/${projectId}`;

  return (
    <main className="mx-auto flex max-w-sm flex-col gap-4 p-6">
      <Card>
        <h1 className="mb-4 text-xl font-semibold">Create an issue</h1>
        <CreateIssueForm projectId={Number(projectId)} basePath={basePath} />
      </Card>
    </main>
  );
}
```

WHY THIS WORKS

`/issues/new` as a literal path segment coexists safely with `/issues/[issueId]` -- Next.js always prefers a literal segment match over a dynamic one at the same level, so a real numeric issue id can never collide with the literal word `new`.

COMMANDS

BASH

```bash
npx tsc --noEmit && npm run build
```

VERIFY

BASH

```
curl -s --cookie "session=$TOKEN" http://localhost:3001/workspaces/1/projects/1/issues/new -o /dev/null -w "%{http_code}\n"
```

EXPECTED RESULT

`tsc` / `build` both exit clean; real `200`. Nothing yet links to this page (LC-09).

LC-09 — CORE: link to the real "New issue" page

Step: `LC-09`

GOAL

A real signed-in user can actually reach `/issues/new` from the real Project Detail page.

TECHNICAL CONTEXT

Pure wiring -- the real last piece connecting LC-03..08 into a real, reachable product flow.

FILE

`web/src/app/(app)/workspaces/[id]/projects/[projectId]/page.tsx`

EXACT LOCATION

Right before the existing `Issues` `<h2>` heading, inside the returned JSX.

BEFORE

TSX

```
      <h2 className="text-sm font-semibold uppercase text-gray-500 dark:text-gray-400">
        Issues
      </h2>
```

IMPLEMENTATION

TSX

```
      <div className="flex items-center gap-4 text-sm">
        <Link
          href={`/workspaces/${id}/projects/${project.id}/issues/new`}
          className="text-blue-600 hover:underline dark:text-blue-400"
```

```
        >
          + New issue
        </Link>
      </div>

      <h2 className="text-sm font-semibold uppercase text-gray-500 dark:text-gray-400">
        Issues
      </h2>
```

AFTER

TSX

```
      <div className="flex items-center gap-4 text-sm">
        <Link
          href={`/workspaces/${id}/projects/${project.id}/issues/new`}
          className="text-blue-600 hover:underline dark:text-blue-400"
        >
          + New issue
        </Link>
      </div>

      <h2 className="text-sm font-semibold uppercase text-gray-500 dark:text-gray-400">
        Issues
      </h2>
```

WHY THIS WORKS

`Link`, already imported at the top of this real file for the real issue-card links below -- no new import needed.

COMMANDS

BASH

```
npx tsc --noEmit && npm run build
```

VERIFY

BASH

```
curl -s --cookie "session=$TOKEN" http://localhost:3001/workspaces/1/projects/1 | grep -o
"New issue"
```

EXPECTED RESULT

`tsc` / `build` both exit clean; the real link appears and navigates to `/issues/new`.

LC-10 — CORE: `updateIssueAction` -- the real generic skeleton

Step: `LC-10`

GOAL

A real new export exists that can update any real subset of an issue's own fields -- not yet called from anywhere.

TECHNICAL CONTEXT

What new problem are you solving? `IssueEditPanel` (LC-12..13) will need to save ONE field at a time (title, then separately status, then later priority, assignee, story points) without a new Server Action per field. `updateIssueAction` is written generically from the very start -- `data: Record<string, unknown>` -- so every future field this app ever adds to the panel reuses this exact same function, unchanged.

FILE

`web/src/lib/actions/issues.ts`

EXACT LOCATION

A brand-new export, added at the end of the file, after `createIssueAction`.

BEFORE

TSX

```
  const issue: { id: number } = await res.json();
  redirect(`${basePath}/issues/${issue.id}`);
}
```

IMPLEMENTATION

TSX

```
  const issue: { id: number } = await res.json();
  redirect(`${basePath}/issues/${issue.id}`);
}

// The ONE real generic mutation every real IssueEditPanel field calls -- never a new Server
// Action per field. `data` carries exactly the one (or few) real field(s) that single edit
// touched (e.g. `{ title: "..." }`, or later `{ priority: "high" }`), matching the real
backend's
```

```
// own `UpdateIssueDto`, which already accepts any subset of its own real fields.
export async function updateIssueAction(
  issueId: number,
  basePath: string,
  data: Record<string, unknown>,
) {
  const token = await requireSessionToken();
}
```

AFTER

TSX

```
  const issue: { id: number } = await res.json();
  redirect(`${basePath}/issues/${issue.id}`);
}

// The ONE real generic mutation every real IssueEditPanel field calls -- never a new Server
// Action per field. `data` carries exactly the one (or few) real field(s) that single edit
// touched (e.g. `{ title: "..." }`, or later `{ priority: "high" }`), matching the real
backend's
// own `UpdateIssueDto`, which already accepts any subset of its own real fields.
export async function updateIssueAction(
  issueId: number,
  basePath: string,
  data: Record<string, unknown>,
) {
  const token = await requireSessionToken();
}
```

WHY THIS WORKS

`data: Record<string, unknown>`, not a typed `Partial<Issue>` -- the real backend's own `UpdateIssueDto` is the real source of truth for which fields are valid; this action is a thin, generic pass-through, never a second place that would need updating every time a new field is added.

VERIFY

BASH

```
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. Nothing calls the real backend yet (LC-11).

LC-11 — CORE: `updateIssueAction` -- the real backend call

Step: `LC-11`

GOAL

The real generic update function actually calls the real backend and refreshes the real page that shows the result.

TECHNICAL CONTEXT

Pure, mechanical continuation of LC-10 -- the real skeleton already validates nothing (by design, it is a thin pass-through); this step adds the real `PATCH` call and the real `revalidatePath`.

FILE

`web/src/lib/actions/issues.ts`

EXACT LOCATION

One real new import at the top of the file; the body of `updateIssueAction`, replacing the single `const token = ...` line.

BEFORE

TSX

```
import { redirect } from "next/navigation";
import { apiFetch } from "@/lib/api";
```

TSX

```
) {
  const token = await requireSessionToken();
}
```

IMPLEMENTATION

TSX

```
import { redirect } from "next/navigation";
import { revalidatePath } from "next/cache";
import { apiFetch } from "@/lib/api";
```

TSX

```
) {
  const token = await requireSessionToken();
```

```
  const res = await apiFetch(`/issues/${issueId}`, {
    method: "PATCH",
    token,
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(data),
  });

  if (!res.ok) {
    throw new Error("Could not update the issue.");
  }

  revalidatePath(`${basePath}/issues/${issueId}`);
  revalidatePath(`${basePath}/board`);
  return res.json();
}
```

AFTER

TSX

```
import { redirect } from "next/navigation";
import { revalidatePath } from "next/cache";
import { apiFetch } from "@/lib/api";
```

TSX

```
) {
  const token = await requireSessionToken();

  const res = await apiFetch(`/issues/${issueId}`, {
    method: "PATCH",
    token,
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(data),
  });

  if (!res.ok) {
    throw new Error("Could not update the issue.");
  }

  revalidatePath(`${basePath}/issues/${issueId}`);
  revalidatePath(`${basePath}/board`);
  return res.json();
}
```

WHY THIS WORKS

A thrown `Error`, not a returned `{ error }` state -- unlike `createIssueAction`'s own form submission, callers of `updateIssueAction` (LC-14's inline field editor, LC-30's board drag handler) call it directly, not through `useActionState`; a thrown error is what lets a caller's own `try`/`catch` (or `.catch()`) react to a real failure.

Two real `revalidatePath` calls, not one -- a real status change made from the board itself (LC-30) must also invalidate the real Issue Detail page, and vice versa, since either real view can show a now-stale status otherwise.

VERIFY

BASH

```
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. Nothing calls `updateIssueAction` from anywhere yet (LC-14).

LC-12 — CORE: `IssueEditPanel` -- the real field type

Step: `LC-12`

GOAL

A real file exists declaring the one real generic row type every editable Issue field will use -- no real component yet.

TECHNICAL CONTEXT

What new problem are you solving? Every real Issue field a student will ever make editable (title now; status next step; priority/assignee/story points in homework) needs the identical real per-row shape. Declaring that real shape once, generically, means a later field is a one-line change to a `fields` array, never a new component.

FILE

`web/src/components/issues/IssueEditPanel.tsx`

EXACT LOCATION

A brand-new file at `web/src/components/issues/IssueEditPanel.tsx`, with no real component yet.

BEFORE

File does not exist yet.

IMPLEMENTATION

TSX

```tsx
"use client";

import type { Issue } from "@/lib/types";

export type EditableField = { key: keyof Issue; label: string; type: "text" };
```

AFTER

TSX

```tsx
"use client";

import type { Issue } from "@/lib/types";

export type EditableField = { key: keyof Issue; label: string; type: "text" };
```

WHY THIS WORKS

A dedicated type alias, exported -- the real call site (LC-19) and every later field addition reference it by name, never a real inline object type repeated at each call site.

VERIFY

BASH

```bash
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. The real component itself comes next (LC-13).

LC-13 — CORE: `IssueEditPanel` -- the real generic wrapper

Step: `LC-13`

GOAL

A real component exists that maps a real `fields` array to one real row per field -- each row still a real empty placeholder.

TECHNICAL CONTEXT

Pure, mechanical continuation of LC-12 -- the real row type already exists; this step adds the real mapping that uses it.

FILE

`web/src/components/issues/IssueEditPanel.tsx`

EXACT LOCATION

The real `IssueEditPanel` / `EditableRow` functions, added right after the real `EditableField` type.

BEFORE

TSX

```tsx
export type EditableField = { key: keyof Issue; label: string; type: "text" };
```

IMPLEMENTATION

TSX

```tsx
export type EditableField = { key: keyof Issue; label: string; type: "text" };

export function IssueEditPanel({
  issue,
  basePath,
  fields,
}: {
  issue: Issue;
  basePath: string;
  fields: EditableField[];
}) {
  return (
    <div className="flex flex-col gap-3">
      {fields.map((field) => (
        <EditableRow key={String(field.key)} issue={issue} basePath={basePath} field={field}
/>
      ))}
    </div>
  );
```

```
}

function EditableRow(_props: { issue: Issue; basePath: string; field: EditableField }) {
  return null;
}
```

AFTER

TSX

```
export type EditableField = { key: keyof Issue; label: string; type: "text" };

export function IssueEditPanel({
  issue,
  basePath,
  fields,
}: {
  issue: Issue;
  basePath: string;
  fields: EditableField[];
}) {
  return (
    <div className="flex flex-col gap-3">
      {fields.map((field) => (
        <EditableRow key={String(field.key)} issue={issue} basePath={basePath} field={field}
/>
      ))}
    </div>
  );
}

function EditableRow(_props: { issue: Issue; basePath: string; field: EditableField }) {
  return null;
}
```

WHY THIS WORKS

`key={String(field.key)}` -- a real stable key per row, so React never confuses one real field's own row state for another's if the `fields` array itself ever reorders.

VERIFY

BASH

```
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. The real row content comes next (LC-14).

LC-14 — CORE: `IssueEditPanel` -- the real row state and save

Step: `LC-14`

GOAL

Each real row now holds its own real local value and calls the real `updateIssueAction` on save -- no real input renders yet.

TECHNICAL CONTEXT

Pure, mechanical continuation of LC-13 -- the real wrapper already maps `fields` to rows; this step gives each row its own real state and its own real save call.

FILE

`web/src/components/issues/IssueEditPanel.tsx`

EXACT LOCATION

One real new import; `EditableRow`'s own real body, replacing its placeholder `return null;`.

BEFORE

TSX

```tsx
import type { Issue } from "@/lib/types";
```

TSX

```tsx
function EditableRow(_props: { issue: Issue; basePath: string; field: EditableField }) {
  return null;
}
```

IMPLEMENTATION

TSX

```tsx
import { useState } from "react";
import type { Issue } from "@/lib/types";
import { updateIssueAction } from "@/lib/actions/issues";
```

TSX

```tsx
function EditableRow({
  issue,
  basePath,
  field,
}: {
  issue: Issue;
  basePath: string;
  field: EditableField;
}) {
  const [value, setValue] = useState(String(issue[field.key] ?? ""));
  const [pending, setPending] = useState(false);

  async function save(newValue: string) {
    setValue(newValue);
    setPending(true);
    try {
      await updateIssueAction(issue.id, basePath, { [field.key]: newValue });
    } finally {
      setPending(false);
    }
  }

  return null;
}
```

AFTER

TSX

```tsx
import { useState } from "react";
import type { Issue } from "@/lib/types";
import { updateIssueAction } from "@/lib/actions/issues";
```

TSX

```tsx
function EditableRow({
  issue,
  basePath,
  field,
}: {
  issue: Issue;
  basePath: string;
  field: EditableField;
}) {
  const [value, setValue] = useState(String(issue[field.key] ?? ""));
  const [pending, setPending] = useState(false);
```

```
  async function save(newValue: string) {
    setValue(newValue);
    setPending(true);
    try {
      await updateIssueAction(issue.id, basePath, { [field.key]: newValue });
    } finally {
      setPending(false);
    }
  }

  return null;
}
```

WHY THIS WORKS

`onBlur`, not `onChange`, will trigger the real save (LC-15) -- a real network call on every keystroke would be wasteful; `onChange` only updates the real local `value` so the input stays responsive.

VERIFY

BASH

```
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. The real input itself comes next (LC-15).

LC-15 — CORE: `IssueEditPanel` -- the real text input

Step: `LC-15`

GOAL

Each real row now actually renders a real, working text input.

TECHNICAL CONTEXT

Pure, mechanical continuation of LC-14 -- the real state and save already exist; this step only renders the real control that drives them.

FILE

`web/src/components/issues/IssueEditPanel.tsx`

EXACT LOCATION

The placeholder `return null;` replaced by the real labeled input.

BEFORE

TSX

```
  return null;
}
```

IMPLEMENTATION

TSX

```
  return (
    <label className="flex flex-col gap-1 text-sm">
      <span className="text-gray-500 dark:text-gray-400">{field.label}</span>
      <input
        type="text"
        value={value}
        disabled={pending}
        onChange={(e) => setValue(e.target.value)}
        onBlur={(e) => save(e.target.value)}
        className="rounded-md border border-gray-300 px-3 py-1 dark:border-gray-600 dark:bg-
gray-800"
      />
    </label>
  );
}
```

AFTER

TSX

```
  return (
    <label className="flex flex-col gap-1 text-sm">
      <span className="text-gray-500 dark:text-gray-400">{field.label}</span>
      <input
        type="text"
        value={value}
        disabled={pending}
        onChange={(e) => setValue(e.target.value)}
        onBlur={(e) => save(e.target.value)}
        className="rounded-md border border-gray-300 px-3 py-1 dark:border-gray-600 dark:bg-
gray-800"
      />
    </label>
```

```
  );
}
```

WHY THIS WORKS

`disabled={pending}` -- while a real save is in flight, the real input is briefly non-editable, so a second real edit cannot race the first.

VERIFY

BASH

```
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. Nothing renders this component yet (LC-19), and the real second field kind ("select") comes next (LC-16).

LC-16 — CORE: `IssueEditPanel` -- the real "select" variant

Step: `LC-16`

GOAL

The real `EditableField` type now allows a second real kind (a dropdown) -- no real rendering for it yet.

TECHNICAL CONTEXT

Pure, mechanical continuation of LC-15 -- the real type grows a second real variant before the real render branch that handles it exists.

FILE

`web/src/components/issues/IssueEditPanel.tsx`

EXACT LOCATION

The `EditableField` type's own real declaration, widened to a real union.

BEFORE

TSX

```
export type EditableField = { key: keyof Issue; label: string; type: "text" };
```

IMPLEMENTATION

TSX

```
export type EditableField =
  | { key: keyof Issue; label: string; type: "text" }
  | { key: keyof Issue; label: string; type: "select"; options: { value: string; label:
string }[] };
```

AFTER

TSX

```
export type EditableField =
  | { key: keyof Issue; label: string; type: "text" }
  | { key: keyof Issue; label: string; type: "select"; options: { value: string; label:
string }[] };
```

WHY THIS WORKS

The real type grows first, deliberately ahead of the real render branch that handles it (LC-17) -- a real "select" field cannot yet be passed in `fields` by any real caller, so this is a safe, inert real addition.

VERIFY

BASH

```
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. The real render branch comes next (LC-17).

LC-17 — CORE: `IssueEditPanel` -- the real "select" branch shape

Step: `LC-17`

GOAL

The real row's own returned JSX now branches on `field.type` -- the real "select" branch still a placeholder.

TECHNICAL CONTEXT

Pure, mechanical continuation of LC-16 -- the real type already allows "select"; this step adds the real branch shape, content still a placeholder.

FILE

`web/src/components/issues/IssueEditPanel.tsx`

EXACT LOCATION

The real `<input>` wrapped in a real `field.type === "text" ? ... : ...` conditional.

BEFORE

TSX

```
      <input
        type="text"
        value={value}
        disabled={pending}
        onChange={(e) => setValue(e.target.value)}
        onBlur={(e) => save(e.target.value)}
        className="rounded-md border border-gray-300 px-3 py-1 dark:border-gray-600 dark:bg-
gray-800"
      />
    </label>
  );
}
```

IMPLEMENTATION

TSX

```
      {field.type === "text" ? (
        <input
          type="text"
          value={value}
          disabled={pending}
          onChange={(e) => setValue(e.target.value)}
          onBlur={(e) => save(e.target.value)}
          className="rounded-md border border-gray-300 px-3 py-1 dark:border-gray-600
```

```
dark:bg-gray-800"
        />
      ) : null}
    </label>
  );
}
```

AFTER

TSX

```
      {field.type === "text" ? (
        <input
          type="text"
          value={value}
          disabled={pending}
          onChange={(e) => setValue(e.target.value)}
          onBlur={(e) => save(e.target.value)}
          className="rounded-md border border-gray-300 px-3 py-1 dark:border-gray-600
dark:bg-gray-800"
        />
      ) : null}
    </label>
  );
}
```

WHY THIS WORKS

The real branch shape is written first, with a real placeholder `null` -- once the real `<select>` content fills it (LC-18), the wrapping structure itself never needs to change again.

VERIFY

BASH

```
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. The real `<select>` content comes next (LC-18).

LC-18 — CORE: `IssueEditPanel` -- the real `<select>` content

Step: `LC-18`

GOAL

The real "select" branch now actually renders a real, working dropdown.

TECHNICAL CONTEXT

Pure, mechanical continuation of LC-17 -- the real branch shape already exists; this step fills its own real placeholder.

FILE

`web/src/components/issues/IssueEditPanel.tsx`

EXACT LOCATION

The real placeholder `null`, replaced by the real `<select>` content.

BEFORE

TSX

```
      ) : null}
```

IMPLEMENTATION

TSX

```
      ) : (
        <select
          value={value}
          disabled={pending}
          onChange={(e) => save(e.target.value)}
          className="rounded-md border border-gray-300 px-2 py-1 dark:border-gray-600
dark:bg-gray-800"
        >
          {field.options.map((opt) => (
            <option key={opt.value} value={opt.value}>
              {opt.label}
            </option>
          ))}
        </select>
      )}
```

AFTER

TSX

```
      ) : (
        <select
          value={value}
          disabled={pending}
          onChange={(e) => save(e.target.value)}
          className="rounded-md border border-gray-300 px-2 py-1 dark:border-gray-600
dark:bg-gray-800"
        >
          {field.options.map((opt) => (
            <option key={opt.value} value={opt.value}>
              {opt.label}
            </option>
          ))}
        </select>
      )}
```

WHY THIS WORKS

A `<select>` saves `onChange`, not `onBlur` -- unlike free text, a dropdown's own real value only ever changes by a deliberate pick; there is no "still typing" state worth waiting out.

VERIFY

BASH

```
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. Nothing renders this component yet (LC-19).

LC-19 — CORE: wire `IssueEditPanel` into Issue Detail

Step: `LC-19`

GOAL

Issue Detail's own title and status are now really editable in place.

TECHNICAL CONTEXT

Pure wiring -- the real panel already exists (LC-12..18); this step replaces the static title/status display with it.

FILE

`web/src/app/(app)/workspaces/[id]/projects/[projectId]/issues/[issueId]/page.tsx`

EXACT LOCATION

The `StatusBadge` import is replaced by `IssueEditPanel`; the static title/badge `<div>` inside `<Card>` is replaced by the real panel.

BEFORE

TSX

```tsx
import { Card } from "@/components/ui/Card";
import { StatusBadge } from "@/components/ui/Badge";
```

TSX

```tsx
      <Card>
        <div className="mb-2 flex items-center justify-between">
          <h1 className="text-xl font-semibold">{issue.title}</h1>
          <StatusBadge status={issue.status} />
        </div>
        <p className="text-sm text-gray-500 dark:text-gray-400">
          {issue.description}
        </p>
```

IMPLEMENTATION

TSX

```tsx
import { Card } from "@/components/ui/Card";
import { IssueEditPanel } from "@/components/issues/IssueEditPanel";
```

TSX

```tsx
      <Card>
        <IssueEditPanel
          issue={issue}
          basePath={`/workspaces/${id}/projects/${projectId}`}
          fields={[
            { key: "title", label: "Title", type: "text" },
            {
              key: "status",
              label: "Status",
              type: "select",
              options: [
                { value: "todo", label: "To do" },
                { value: "in_progress", label: "In progress" },
```

```
                { value: "review", label: "Review" },
                { value: "done", label: "Done" },
              ],
            },
          ]}
        />
        <p className="mt-3 text-sm text-gray-500 dark:text-gray-400">
          {issue.description}
        </p>
```

AFTER

TSX

```
import { Card } from "@/components/ui/Card";
import { IssueEditPanel } from "@/components/issues/IssueEditPanel";
```

TSX

```
      <Card>
        <IssueEditPanel
          issue={issue}
          basePath={`/workspaces/${id}/projects/${projectId}`}
          fields={[
            { key: "title", label: "Title", type: "text" },
            {
              key: "status",
              label: "Status",
              type: "select",
              options: [
                { value: "todo", label: "To do" },
                { value: "in_progress", label: "In progress" },
                { value: "review", label: "Review" },
                { value: "done", label: "Done" },
              ],
            },
          ]}
        />
        <p className="mt-3 text-sm text-gray-500 dark:text-gray-400">
          {issue.description}
        </p>
```

WHY THIS WORKS

`fields` is written inline, at the real call site -- not hard-coded inside `IssueEditPanel` itself -- so a later real caller (none exists yet, but the pattern already allows it) could show a different real subset of fields without touching the component at all.

COMMANDS

BASH

```
npx tsc --noEmit && npm run build
```

VERIFY

BASH

```
TOKEN=$(curl -s -X POST http://localhost:3000/auth/login -H "Content-Type: application/json"
\
  -d '{"email":"lina.owner@b4fboard.dev","password":"Password123!"}' | python3 -c \
  "import sys,json;print(json.load(sys.stdin)['accessToken'])")
curl -s -X PATCH http://localhost:3000/issues/1 -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" -d '{"status":"in_progress"}' | python3 -c \
  "import sys,json;print(json.load(sys.stdin)['status'])"
```

EXPECTED RESULT

`tsc` / `build` both exit clean; real `in_progress` printed -- the identical real call the panel's own status `<select>` makes. The real board comes next (LC-20..33).

ACTUAL CLASSROOM RESULT FOR LC-19

The classroom implementation retained the existing title/status summary and added description to the editable fields. Apply this exact reconciliation before LC-20; this records the delivered tree rather than silently replacing it with the rehearsal variant.

TSX

```
import { Card } from "@/components/ui/Card";
import { StatusBadge } from "@/components/ui/Badge";
import { IssueEditPanel } from "@/components/issues/IssueEditPanel";
```

TSX

```
      <Card>
        <div className="mb-2 flex items-center justify-between">
          <h1 className="text-xl font-semibold">{issue.title}</h1>
          <StatusBadge status={issue.status} />
        </div>
        <IssueEditPanel
          issue={issue}
          basePath={`/workspaces/${id}/projects/${projectId}`}
          fields={[
            { key: "title", label: "Title", type: "text" },
            { key: "description", label: "Description", type: "text" },
            {
```

```
            key: "status",
            label: "Status",
            type: "select",
            options: [
              { value: "todo", label: "To do" },
              { value: "in_progress", label: "In progress" },
              { value: "review", label: "Review" },
              { value: "done", label: "Done" },
            ],
          },
        ]}
      />
      <p className="text-sm text-gray-500 dark:text-gray-400">
        {issue.description}
      </p>
```

LC-20 — CORE: install `@dnd-kit/core`

Step: `LC-20`

GOAL

The real `@dnd-kit/core` package is installed -- this program's genuine first real drag-and-drop mechanism.

TECHNICAL CONTEXT

This program always installs a real new package right before its own first real use, the identical real discipline this program used for Tailwind (Session 24) and Redux Toolkit (Session 25) -- the real justified case exists now: a real persisted board needs real pointer- driven drag-and-drop, which this app has never needed before.

FILE

`web/package.json`

EXACT LOCATION

Run the real install; `npm` itself edits the `"dependencies"` block.

BEFORE

JSON

```
  "dependencies": {
    "@reduxjs/toolkit": "^2.14.0",
```

```
    "next": "15.5.25",
    "react": "19.1.0",
    "react-dom": "19.1.0",
    "react-redux": "^9.3.1",
    "server-only": "^0.0.1"
  },
```

IMPLEMENTATION

JSON

```
    "@dnd-kit/core": "^6.3.1",
```

AFTER

JSON

```
  "dependencies": {
    "@dnd-kit/core": "^6.3.1",
    "@reduxjs/toolkit": "^2.14.0",
    "next": "15.5.25",
    "react": "19.1.0",
    "react-dom": "19.1.0",
    "react-redux": "^9.3.1",
    "server-only": "^0.0.1"
  },
```

WHY THIS WORKS

`@dnd-kit/core` only, not a higher-level drag-and-drop framework with built-in sorting/lists -- this app's own real board only ever needs one draggable card moving between droppable columns, never a reorderable list within a column.

COMMANDS

BASH

```
cd web
npm install @dnd-kit/core@6.3.1
```

VERIFY

BASH

```
grep '"@dnd-kit/core"' package.json
```

EXPECTED RESULT

The real dependency appears.

LC-21 — CORE: `IssueCard` -- the real draggable skeleton

Step: `LC-21`

GOAL

A real component exists that renders one issue as a real draggable element -- no real title link yet.

TECHNICAL CONTEXT

What new problem are you solving? `useDraggable` is `@dnd-kit`'s own real hook for "this element can be picked up" -- it hands back real `listeners` / `attributes` to spread onto the element, a real `transform` to visually follow the pointer while dragging, and a real `isDragging` flag.

FILE

`web/src/components/board/IssueCard.tsx`

EXACT LOCATION

A brand-new file at `web/src/components/board/IssueCard.tsx`, in a new `components/board/` folder.

BEFORE

File does not exist yet.

IMPLEMENTATION

TSX

```tsx
"use client";

import { useDraggable } from "@dnd-kit/core";
import type { Issue } from "@/lib/types";

export function IssueCard({ issue, href }: { issue: Issue; href: string }) {
  const { attributes, listeners, setNodeRef, transform, isDragging } = useDraggable({
    id: issue.id,
  });

  const style = transform
```

```
    ? { transform: `translate3d(${transform.x}px, ${transform.y}px, 0)` }
    : undefined;

  return (
    <div
      ref={setNodeRef}
      style={style}
      {...listeners}
      {...attributes}
      className={`cursor-grab rounded-md border border-gray-200 bg-white p-2 text-sm shadow-
sm active:cursor-grabbing dark:border-gray-700 dark:bg-gray-900 ${
        isDragging ? "opacity-50" : ""
      }`}
    >
      {issue.key}
    </div>
  );
}
```

AFTER

TSX

```
"use client";

import { useDraggable } from "@dnd-kit/core";
import type { Issue } from "@/lib/types";

export function IssueCard({ issue, href }: { issue: Issue; href: string }) {
  const { attributes, listeners, setNodeRef, transform, isDragging } = useDraggable({
    id: issue.id,
  });

  const style = transform
    ? { transform: `translate3d(${transform.x}px, ${transform.y}px, 0)` }
    : undefined;

  return (
    <div
      ref={setNodeRef}
      style={style}
      {...listeners}
      {...attributes}
      className={`cursor-grab rounded-md border border-gray-200 bg-white p-2 text-sm shadow-
sm active:cursor-grabbing dark:border-gray-700 dark:bg-gray-900 ${
        isDragging ? "opacity-50" : ""
      }`}
```

```
    >
      {issue.key}
    </div>
  );
}
```

WHY THIS WORKS

`id: issue.id` -- `useDraggable` needs a real, stable, unique id per draggable; the real `onDragEnd` handler (LC-30) reads this exact id back off `event.active.id` to know which real issue just moved.

VERIFY

BASH

```
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. The real title link comes next (LC-22); nothing renders this component yet (LC-26).

LC-22 — CORE: `IssueCard` -- the real title link

Step: `LC-22`

GOAL

The real card shows its own real key and title, linking to the real Issue Detail page.

TECHNICAL CONTEXT

Pure, mechanical continuation of LC-21 -- the real draggable shell already exists; this step only fills its own real content.

FILE

`web/src/components/board/IssueCard.tsx`

EXACT LOCATION

One real new import; the card's own plain `{issue.key}` text, replaced by a real `<Link>`.

BEFORE

TSX

```
import { useDraggable } from "@dnd-kit/core";
```

TSX

```
      {issue.key}
    </div>
```

IMPLEMENTATION

TSX

```
import Link from "next/link";
import { useDraggable } from "@dnd-kit/core";
```

TSX

```
      <Link
        href={href}
        className="font-medium hover:underline"
        onPointerDown={(e) => e.stopPropagation()}
      >
        <span className="text-gray-400">{issue.key}</span> {issue.title}
      </Link>
    </div>
```

AFTER

TSX

```
import Link from "next/link";
import { useDraggable } from "@dnd-kit/core";
```

TSX

```
      <Link
        href={href}
        className="font-medium hover:underline"
        onPointerDown={(e) => e.stopPropagation()}
      >
        <span className="text-gray-400">{issue.key}</span> {issue.title}
      </Link>
    </div>
```

WHY THIS WORKS

`onPointerDown={(e) => e.stopPropagation()}` -- without it, clicking the title would also register as the start of a drag under `@dnd-kit`'s own pointer sensor.

VERIFY

BASH

```
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. Nothing renders this component yet (LC-26).

LC-23 — CORE: `BoardColumn` -- the real droppable element

Step: `LC-23`

GOAL

A real component exists that renders one real drop target -- no real header, no real card list yet.

TECHNICAL CONTEXT

`useDroppable` is `@dnd-kit`'s own real counterpart to `useDraggable` -- "a dragged element may be dropped here." The column's own real `status` string doubles as its own real droppable id, so `onDragEnd` (LC-30) can read the real new status straight off `event.over.id`.

FILE

`web/src/components/board/BoardColumn.tsx`

EXACT LOCATION

A brand-new file at `web/src/components/board/BoardColumn.tsx`, a sibling of `IssueCard.tsx`.

BEFORE

File does not exist yet.

IMPLEMENTATION

TSX

```tsx
"use client";

import { useDroppable } from "@dnd-kit/core";

export function BoardColumn({ status }: { status: string }) {
  const { setNodeRef, isOver } = useDroppable({ id: status });

  return (
    <div
      ref={setNodeRef}
      className={`flex min-h-20 flex-col gap-2 rounded-lg p-1 transition-colors ${
        isOver ? "bg-blue-50 dark:bg-blue-950/40" : ""
      }`}
    />
  );
}
```

AFTER

TSX

```tsx
"use client";

import { useDroppable } from "@dnd-kit/core";

export function BoardColumn({ status }: { status: string }) {
  const { setNodeRef, isOver } = useDroppable({ id: status });

  return (
    <div
      ref={setNodeRef}
      className={`flex min-h-20 flex-col gap-2 rounded-lg p-1 transition-colors ${
        isOver ? "bg-blue-50 dark:bg-blue-950/40" : ""
      }`}
    />
  );
}
```

WHY THIS WORKS

`isOver` drives a real, purely visual highlight -- it never gates whether a drop is accepted; `@dnd-kit` always fires `onDragEnd` with whatever column the pointer was over.

VERIFY

BASH

```
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. The real remaining props come next (LC-24); the real header and outer wrapper after that (LC-25).

LC-24 — CORE: `BoardColumn` -- the real props it will need

Step: `LC-24`

GOAL

The real component's own props signature grows to the real final shape it will need -- no real new behavior yet.

TECHNICAL CONTEXT

Pure, mechanical continuation of LC-23 -- the real header (LC-25) and real card list (LC-26) both need `label` / `issues` / `basePath`; declaring them now, together, avoids a real signature edit per field later.

FILE

`web/src/components/board/BoardColumn.tsx`

EXACT LOCATION

One real new import; the real props signature grows from `{ status }` to its real final shape.

BEFORE

TSX

```
import { useDroppable } from "@dnd-kit/core";
```

TSX

```
export function BoardColumn({ status }: { status: string }) {
```

IMPLEMENTATION

TSX

```tsx
import { useDroppable } from "@dnd-kit/core";
import type { Issue } from "@/lib/types";
```

TSX

```tsx
export function BoardColumn({
  status,
  label,
  issues,
  basePath,
}: {
  status: string;
  label: string;
  issues: Issue[];
  basePath: string;
}) {
```

AFTER

TSX

```tsx
import { useDroppable } from "@dnd-kit/core";
import type { Issue } from "@/lib/types";
```

TSX

```tsx
export function BoardColumn({
  status,
  label,
  issues,
  basePath,
}: {
  status: string;
  label: string;
  issues: Issue[];
  basePath: string;
}) {
```

WHY THIS WORKS

`issues: Issue[]`, not a real count passed separately -- the real card list (LC-26) needs the real array itself, not just its length, so the real header's own `issues.length` (LC-25) reuses the identical real prop rather than a second, redundant one.

VERIFY

BASH

```
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. `label` / `issues` / `basePath` are not read from anywhere yet (LC-25/26).

LC-25 — CORE: `BoardColumn` -- the real header and outer wrapper

Step: `LC-25`

GOAL

The real column now shows its own real label and issue count, wrapped with the real droppable element inside it.

TECHNICAL CONTEXT

Pure, mechanical continuation of LC-24 -- the real props already exist; this step only adds the real surrounding layout.

FILE

`web/src/components/board/BoardColumn.tsx`

EXACT LOCATION

The returned droppable `<div>` wrapped in a real outer `<div>` with a real `<h3>` header above it.

BEFORE

TSX

```
  return (
    <div
      ref={setNodeRef}
      className={`flex min-h-20 flex-col gap-2 rounded-lg p-1 transition-colors ${
        isOver ? "bg-blue-50 dark:bg-blue-950/40" : ""
      }`}
    />
  );
```

IMPLEMENTATION

TSX

```tsx
  return (
    <div className="flex w-64 flex-shrink-0 flex-col gap-2">
      <h3 className="text-sm font-semibold text-gray-600 dark:text-gray-300">
        {label} <span className="text-gray-400">({issues.length})</span>
      </h3>
      <div
        ref={setNodeRef}
        className={`flex min-h-20 flex-col gap-2 rounded-lg p-1 transition-colors ${
          isOver ? "bg-blue-50 dark:bg-blue-950/40" : ""
        }`}
      />
    </div>
  );
```

AFTER

TSX

```tsx
  return (
    <div className="flex w-64 flex-shrink-0 flex-col gap-2">
      <h3 className="text-sm font-semibold text-gray-600 dark:text-gray-300">
        {label} <span className="text-gray-400">({issues.length})</span>
      </h3>
      <div
        ref={setNodeRef}
        className={`flex min-h-20 flex-col gap-2 rounded-lg p-1 transition-colors ${
          isOver ? "bg-blue-50 dark:bg-blue-950/40" : ""
        }`}
      />
    </div>
  );
```

WHY THIS WORKS

`w-64 flex-shrink-0` lives on the real OUTER `<div>`, not the droppable one -- the real column's own fixed width must never shrink in the real horizontally-scrolling board, while the real droppable area inside it still fills the available real height.

VERIFY

BASH

```bash
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. The real card list comes next (LC-26).

LC-26 — CORE: `BoardColumn` -- the real card list

Step: `LC-26`

GOAL

The real column now lists its own real `IssueCard`s, with a real empty state.

TECHNICAL CONTEXT

Pure, mechanical continuation of LC-25 -- the real droppable shell already exists; this step fills its own real children.

FILE

`web/src/components/board/BoardColumn.tsx`

EXACT LOCATION

One real new import; the droppable `<div>`'s own self-closing tag grows real children.

BEFORE

TSX

```
import type { Issue } from "@/lib/types";
```

TSX

```
          isOver ? "bg-blue-50 dark:bg-blue-950/40" : ""
        }`}
      />
```

IMPLEMENTATION

TSX

```
import type { Issue } from "@/lib/types";
import { IssueCard } from "./IssueCard";
```

TSX

```
          isOver ? "bg-blue-50 dark:bg-blue-950/40" : ""
        }`}
      >
```

```
        {issues.map((issue) => (
          <IssueCard key={issue.id} issue={issue} href={`${basePath}/issues/${issue.id}`} />
        ))}
        {issues.length === 0 && (
          <p className="rounded-md border border-dashed border-gray-200 p-3 text-center
text-xs text-gray-400 dark:border-gray-800">
            No issues
          </p>
        )}
      </div>
```

AFTER

TSX

```
import type { Issue } from "@/lib/types";
import { IssueCard } from "./IssueCard";
```

TSX

```
          isOver ? "bg-blue-50 dark:bg-blue-950/40" : ""
        }`}
      >
        {issues.map((issue) => (
          <IssueCard key={issue.id} issue={issue} href={`${basePath}/issues/${issue.id}`} />
        ))}
        {issues.length === 0 && (
          <p className="rounded-md border border-dashed border-gray-200 p-3 text-center
text-xs text-gray-400 dark:border-gray-800">
            No issues
          </p>
        )}
      </div>
```

WHY THIS WORKS

One real single source of truth list, filtered per column -- a real move between columns is one state update (LC-27), never two separate real column-owned lists.

VERIFY

BASH

```
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. Nothing renders this component yet (LC-29).

LC-27 — CORE: `Board` -- the real local state

Step: `LC-27`

GOAL

A real component exists that holds a real local, optimistic copy of the server's own issue list -- no real drag-and-drop yet.

TECHNICAL CONTEXT

What new problem are you solving? Waiting for a real network round-trip before a dragged card visually lands would feel broken -- real drag-and-drop UIs move the item immediately ("optimistically"). This step builds the real local copy that move will update; LC-28..30 build the real drag mechanism and the real move itself.

FILE

`web/src/components/board/Board.tsx`

EXACT LOCATION

A brand-new file at `web/src/components/board/Board.tsx`, with no real columns yet.

BEFORE

File does not exist yet.

IMPLEMENTATION

TSX

```tsx
"use client";

import { useState } from "react";
import type { Issue } from "@/lib/types";

export function Board({
  issues: serverIssues,
  basePath,
}: {
  issues: Issue[];
  basePath: string;
```

```
}) {
  const [issues, setIssues] = useState(serverIssues);

  return <div>{issues.length} issues</div>;
}
```

AFTER

TSX

```tsx
"use client";

import { useState } from "react";
import type { Issue } from "@/lib/types";

export function Board({
  issues: serverIssues,
  basePath,
}: {
  issues: Issue[];
  basePath: string;
}) {
  const [issues, setIssues] = useState(serverIssues);

  return <div>{issues.length} issues</div>;
}
```

WHY THIS WORKS

`useState(serverIssues)` -- initialized once, from the real server-fetched prop; later real server refetches (e.g. a real page revisit) create a real new component instance, never a stale one silently reused.

VERIFY

BASH

```bash
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. The real columns come next (LC-28).

LC-28 — CORE: `Board` -- the real columns constant

Step: `LC-28`

GOAL

A real, named list of the board's own four real status columns exists -- not yet rendered.

TECHNICAL CONTEXT

Pure, mechanical continuation of LC-27 -- a real, explicit constant, declared once, rather than the four real status strings repeated at every real call site.

FILE

`web/src/components/board/Board.tsx`

EXACT LOCATION

Two real new imports; a real `COLUMNS` constant added above the component.

BEFORE

TSX

```tsx
import { useState } from "react";
import type { Issue } from "@/lib/types";
```

IMPLEMENTATION

TSX

```tsx
import { useState } from "react";
import { DndContext } from "@dnd-kit/core";
import type { Issue } from "@/lib/types";
import { BoardColumn } from "./BoardColumn";

const COLUMNS: { status: string; label: string }[] = [
  { status: "todo", label: "To do" },
  { status: "in_progress", label: "In progress" },
  { status: "review", label: "Review" },
  { status: "done", label: "Done" },
];
```

AFTER

TSX

```tsx
import { useState } from "react";
import { DndContext } from "@dnd-kit/core";
import type { Issue } from "@/lib/types";
```

```
import { BoardColumn } from "./BoardColumn";

const COLUMNS: { status: string; label: string }[] = [
  { status: "todo", label: "To do" },
  { status: "in_progress", label: "In progress" },
  { status: "review", label: "Review" },
  { status: "done", label: "Done" },
];
```

WHY THIS WORKS

The real `status` strings here are the exact same literal strings the real backend's own `Issue.status` column already validates against -- never a second, separate real enum to keep in sync.

VERIFY

BASH

```
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. The real `DndContext` /render comes next (LC-29).

LC-29 — CORE: `Board` -- the real `DndContext` and render

Step: `LC-29`

GOAL

The real board now renders all four real status columns, inside a real `DndContext` -- dropping a card does not yet move it.

TECHNICAL CONTEXT

Pure, mechanical continuation of LC-28 -- the real columns constant already exists; this step renders it, wrapped in the real `@dnd-kit` context every droppable column needs.

FILE

`web/src/components/board/Board.tsx`

EXACT LOCATION

The placeholder `return` replaced by the real `DndContext` /column render.

BEFORE

TSX

```
  return <div>{issues.length} issues</div>;
}
```

IMPLEMENTATION

TSX

```
  return (
    <DndContext>
      <div className="flex gap-4 overflow-x-auto pb-4">
        {COLUMNS.map((col) => (
          <BoardColumn
            key={col.status}
            status={col.status}
            label={col.label}
            issues={issues.filter((i) => i.status === col.status)}
            basePath={basePath}
          />
        ))}
      </div>
    </DndContext>
  );
}
```

AFTER

TSX

```
  return (
    <DndContext>
      <div className="flex gap-4 overflow-x-auto pb-4">
        {COLUMNS.map((col) => (
          <BoardColumn
            key={col.status}
            status={col.status}
            label={col.label}
            issues={issues.filter((i) => i.status === col.status)}
            basePath={basePath}
          />
        ))}
      </div>
    </DndContext>
```

```
  );
}
```

WHY THIS WORKS

Every `BoardColumn` (and, inside it, every `IssueCard`) must render underneath a real `DndContext` -- `useDroppable` / `useDraggable` both need its real surrounding context to do anything at all.

VERIFY

BASH

```
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. Dropping a card does not yet move it (LC-30).

LC-30 — CORE: `Board` -- the real optimistic move

Step: `LC-30`

GOAL

Dragging a real card from one real column to another actually moves it, instantly, in the real UI -- not yet confirmed against the real backend.

TECHNICAL CONTEXT

Pure, mechanical continuation of LC-29 -- the real columns already render; this step adds the real handler `DndContext`'s own `onDragEnd` calls.

FILE

`web/src/components/board/Board.tsx`

EXACT LOCATION

One real new import; a real `handleDragEnd` function added before the `return`; `DndContext`'s own `onDragEnd` prop wired.

BEFORE

TSX

```tsx
import { DndContext } from "@dnd-kit/core";
import type { Issue } from "@/lib/types";
```

TSX

```tsx
  return (
    <DndContext>
```

IMPLEMENTATION

TSX

```tsx
import { DndContext, type DragEndEvent } from "@dnd-kit/core";
import type { Issue } from "@/lib/types";
import { updateIssueAction } from "@/lib/actions/issues";
```

TSX

```tsx
  function handleDragEnd(event: DragEndEvent) {
    const { active, over } = event;
    if (!over) return;

    const issueId = active.id as number;
    const newStatus = over.id as string;
    const issue = issues.find((i) => i.id === issueId);
    if (!issue || issue.status === newStatus) return;

    setIssues((prev) =>
      prev.map((i) => (i.id === issueId ? { ...i, status: newStatus } : i)),
    );

    updateIssueAction(issueId, basePath, { status: newStatus });
  }

  return (
    <DndContext onDragEnd={handleDragEnd}>
```

AFTER

TSX

```tsx
import { DndContext, type DragEndEvent } from "@dnd-kit/core";
import type { Issue } from "@/lib/types";
import { updateIssueAction } from "@/lib/actions/issues";
```

TSX

```tsx
  function handleDragEnd(event: DragEndEvent) {
    const { active, over } = event;
    if (!over) return;

    const issueId = active.id as number;
    const newStatus = over.id as string;
    const issue = issues.find((i) => i.id === issueId);
    if (!issue || issue.status === newStatus) return;

    setIssues((prev) =>
      prev.map((i) => (i.id === issueId ? { ...i, status: newStatus } : i)),
    );

    updateIssueAction(issueId, basePath, { status: newStatus });
  }

  return (
    <DndContext onDragEnd={handleDragEnd}>
```

WHY THIS WORKS

`if (!issue || issue.status === newStatus) return;` -- dropping a card back into the column it already belongs to (or an event with no real match) must do nothing, never an empty/no-op real PATCH call.

VERIFY

BASH

```bash
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. A real failed backend call is not yet rolled back (LC-31).

LC-31 — CORE: `Board` -- the real rollback on a failed move

Step: `LC-31`

GOAL

A real move the backend rejects (e.g. a real `403` -- not this issue's own reporter, assignee, admin, or owner) is visually undone, never left showing a status the server never actually accepted.

TECHNICAL CONTEXT

Pure, mechanical continuation of LC-30 -- the real optimistic move already happens; this step adds the real `.catch()` that reverses it.

FILE

`web/src/components/board/Board.tsx`

EXACT LOCATION

Inside `handleDragEnd`: a real snapshot taken before the optimistic move, and a real `.catch()` added to the existing `updateIssueAction` call.

BEFORE

TSX

```
    setIssues((prev) =>
      prev.map((i) => (i.id === issueId ? { ...i, status: newStatus } : i)),
    );

    updateIssueAction(issueId, basePath, { status: newStatus });
  }
```

IMPLEMENTATION

TSX

```
    const previousIssues = issues;
    setIssues((prev) =>
      prev.map((i) => (i.id === issueId ? { ...i, status: newStatus } : i)),
    );

    updateIssueAction(issueId, basePath, { status: newStatus }).catch(() => {
      setIssues(previousIssues);
    });
  }
```

AFTER

TSX

```
    const previousIssues = issues;
    setIssues((prev) =>
      prev.map((i) => (i.id === issueId ? { ...i, status: newStatus } : i)),
    );
```

```
    updateIssueAction(issueId, basePath, { status: newStatus }).catch(() => {
      setIssues(previousIssues);
    });
  }
```

WHY THIS WORKS

`previousIssues` is captured BEFORE the optimistic `setIssues` call, from the same `issues` closure `handleDragEnd` already has -- capturing it after would already be the new, optimistic value, making the real rollback a no-op.

VERIFY

BASH

```
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. Nothing renders this component yet (LC-32).

LC-32 — CORE: the real Board page -- the real skeleton

Step: `LC-32`

GOAL

A real, visitable page exists that fetches a project's own real data -- no real board yet.

TECHNICAL CONTEXT

The identical real page shape every other project-scoped page already uses: fetch the real project, guard with `notFound()`.

FILE

`web/src/app/(app)/workspaces/[id]/projects/[projectId]/board/page.tsx`

EXACT LOCATION

A brand-new file at `.../projects/[projectId]/board/page.tsx`, with no real board yet.

BEFORE

File does not exist yet.

IMPLEMENTATION

TSX

```tsx
import { notFound } from "next/navigation";
import { apiFetch } from "@/lib/api";
import type { Project } from "@/lib/types";

export default async function BoardPage({
  params,
}: {
  params: Promise<{ id: string; projectId: string }>;
}) {
  const { id, projectId } = await params;

  const projectRes = await apiFetch(`/projects/${projectId}`);
  if (!projectRes.ok) {
    notFound();
  }
  const project: Project = await projectRes.json();

  return <main>{project.key}</main>;
}
```

AFTER

TSX

```tsx
import { notFound } from "next/navigation";
import { apiFetch } from "@/lib/api";
import type { Project } from "@/lib/types";

export default async function BoardPage({
  params,
}: {
  params: Promise<{ id: string; projectId: string }>;
}) {
  const { id, projectId } = await params;

  const projectRes = await apiFetch(`/projects/${projectId}`);
  if (!projectRes.ok) {
    notFound();
  }
```

```
  const project: Project = await projectRes.json();

  return <main>{project.key}</main>;
}
```

WHY THIS WORKS

The real page re-fetches the real project itself, rather than receiving it as a prop -- the identical real convention every Server Component page in this app already follows.

VERIFY

BASH

```
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. The real board itself comes next (LC-33).

LC-33 — CORE: the real Board page -- the real fetch and render

Step: `LC-33`

GOAL

The real page now fetches its own real issues and renders the real board, with a real back link.

TECHNICAL CONTEXT

Pure wiring -- the real `Board` already exists (LC-27..31); this step gives it real data.

FILE

`web/src/app/(app)/workspaces/[id]/projects/[projectId]/board/page.tsx`

EXACT LOCATION

Two real new imports; the issues fetch added right after `project`; the placeholder `return` replaced by the real board.

BEFORE

TSX

```tsx
import { notFound } from "next/navigation";
import { apiFetch } from "@/lib/api";
import type { Project } from "@/lib/types";
```

TSX

```tsx
  const project: Project = await projectRes.json();

  return <main>{project.key}</main>;
}
```

IMPLEMENTATION

TSX

```tsx
import Link from "next/link";
import { notFound } from "next/navigation";
import { apiFetch } from "@/lib/api";
import type { Issue, Project } from "@/lib/types";
import { Board } from "@/components/board/Board";
```

TSX

```tsx
  const project: Project = await projectRes.json();

  const issuesRes = await apiFetch(`/issues?projectId=${project.id}`);
  const issues: Issue[] = await issuesRes.json();

  return (
    <main className="mx-auto flex max-w-4xl flex-col gap-4 p-6">
      <Link
        href={`/workspaces/${id}/projects/${project.id}`}
        className="text-sm text-blue-600 hover:underline dark:text-blue-400"
      >
        Back to project
      </Link>
      <h1 className="text-xl font-semibold">
        {project.key} — Board
      </h1>
      <Board issues={issues} basePath={`/workspaces/${id}/projects/${project.id}`} />
    </main>
  );
}
```

AFTER

TSX

```tsx
import Link from "next/link";
import { notFound } from "next/navigation";
import { apiFetch } from "@/lib/api";
import type { Issue, Project } from "@/lib/types";
import { Board } from "@/components/board/Board";
```

TSX

```tsx
  const project: Project = await projectRes.json();

  const issuesRes = await apiFetch(`/issues?projectId=${project.id}`);
  const issues: Issue[] = await issuesRes.json();

  return (
    <main className="mx-auto flex max-w-4xl flex-col gap-4 p-6">
      <Link
        href={`/workspaces/${id}/projects/${project.id}`}
        className="text-sm text-blue-600 hover:underline dark:text-blue-400"
      >
        Back to project
      </Link>
      <h1 className="text-xl font-semibold">
        {project.key} — Board
      </h1>
      <Board issues={issues} basePath={`/workspaces/${id}/projects/${project.id}`} />
    </main>
  );
}
```

WHY THIS WORKS

`GET /issues?projectId=` -- the identical real route Project Detail's own Issues list already uses; the board is a different real VIEW of the same real data, never a second real data source.

COMMANDS

BASH

```bash
npx tsc --noEmit && npm run build
```

VERIFY

BASH

```
curl -s --cookie "session=$TOKEN" http://localhost:3001/workspaces/1/projects/1/board -o
/dev/null -w "%{http_code}\n"
```

EXPECTED RESULT

`tsc` / `build` both exit clean; real `200`. Nothing yet links to this page (LC-34).

LC-34 — CORE: link to the real Board; LIVE Phase A complete

Step: `LC-34`

GOAL

A real signed-in user can actually reach the real board from Project Detail.

TECHNICAL CONTEXT

Pure wiring -- the real last piece connecting LC-20..33 into a real, reachable product flow.

FILE

`web/src/app/(app)/workspaces/[id]/projects/[projectId]/page.tsx`

EXACT LOCATION

In `web/src/app/(app)/workspaces/[id]/projects/[projectId]/page.tsx`, inside the real link row LC-09 already added, right before the existing "+ New issue" link.

BEFORE

TSX

```
      <div className="flex items-center gap-4 text-sm">
        <Link
          href={`/workspaces/${id}/projects/${project.id}/issues/new`}
          className="text-blue-600 hover:underline dark:text-blue-400"
        >
          + New issue
        </Link>
      </div>
```

IMPLEMENTATION

TSX

```
      <div className="flex items-center gap-4 text-sm">
        <Link
          href={`/workspaces/${id}/projects/${project.id}/board`}
          className="text-blue-600 hover:underline dark:text-blue-400"
        >
          Board
        </Link>
        <Link
          href={`/workspaces/${id}/projects/${project.id}/issues/new`}
          className="text-blue-600 hover:underline dark:text-blue-400"
        >
          + New issue
        </Link>
      </div>
```

AFTER

TSX

```
      <div className="flex items-center gap-4 text-sm">
        <Link
          href={`/workspaces/${id}/projects/${project.id}/board`}
          className="text-blue-600 hover:underline dark:text-blue-400"
        >
          Board
        </Link>
        <Link
          href={`/workspaces/${id}/projects/${project.id}/issues/new`}
          className="text-blue-600 hover:underline dark:text-blue-400"
        >
          + New issue
        </Link>
      </div>
```

WHY THIS WORKS

`Link`, already imported at the top of this real file for the real issue-card links below -- no new import needed.

COMMANDS

BASH

```
npx tsc --noEmit && npm run build
```

VERIFY

BASH

```
curl -s --cookie "session=$TOKEN" http://localhost:3001/workspaces/1/projects/1 | grep -o
"Board"
```

EXPECTED RESULT

`tsc` / `build` both exit clean; the real link appears and navigates to the real board. LIVE Phase A is complete: real issue creation, a real generic two-field editable panel, and a real persisted drag-and-drop board with optimistic update and rollback.